사내 AI 해커톤 · 결과 발표

이음 E:UM

사건은 하나, 도구도 하나

배치 실행·서버 관제·업무 관제를
실행 번호 하나로 잇는 시스템 운영 도구

기획서에서 약속한 세 흐름을 실제로 동작하는 도구로 만들었습니다.

실행 → 연결 → 활용



2026. 11.

한 장 요약

기획서의 약속을 동작하는 도구로 만들었습니다

스케줄러·서버 관제·업무 관제를 실행 파일 하나로 합치고, 실행 번호로 함수 단위까지 이었습니다. 심사 기준 세 가지에 이렇게 답합니다.

완성도 30

핵심 기능이 구현되어 정상적으로 동작하는가

3 / 3

약속한 세 흐름이 모두 동작

- 테스트 883건 통과 · 실패 0
- 개념 완결 E2E 통과(실제 perf)
- 시연 장면을 모두 재실행해 확인

업무 활용성 30

구현된 결과물을 사내 업무에 즉시 적용할 수 있는가

1 · 1 · 1

실행 파일·화면·방화벽 포트

- 폐쇄망, 기존 셸 스크립트 그대로
- 로그인·역할 3개·감사 기록·TLS
- 기존 도구 옆에서 관측부터 시작

개선 효과 30

구현된 결과물의 업무 개선 효과가 구체적이고 타당한가

5 → 2단계

장애 원인 추적 단계

- 재현 없이 함수까지 확인
- 크래시 6초 뒤 원인 보고
- 운영 데이터 5종을 SQL로

서류 심사 때는 **설계 시안·합성 예시**였던 화면과 수치가, 이제 **실제 동작 화면·다시 실행해 잰 값**입니다.

문제

사건은 하나인데, 도구는 여럿입니다

장중 체결 지연 한 건을 따라가려면 도구 네 곳을 거치고, 원인 코드는 부하를 다시 만들어야 찾습니다.

배치 스케줄러 — 상용 · Control-M

무엇을 언제 실행했고, 성공했나

✕

서버 관제 — 상용 · onTune

어느 서버·프로세스가 CPU·메모리를 썼나

✕

업무 관제 — 사내 개발 APM

주문이 몇 건 들어와 얼마나 걸렸나

공통 키가 없어 사람이 시각·서버 이름으로 맞춥니다

어느 코드가 CPU를 썼는지는 어느 도구에도 남지 않습니다

장중 체결 지연을 지금 따라가는 순서

1	**업무 관제**	응답시간 상승, 처리 대기 증가 확인
2	**운영 담당**	해당 서비스의 서버·프로세스 확인
3	**서버 관제**	해당 시각 CPU 확인. "CPU를 다 씀"에서 멈춤
4	**스케줄러**	서비스를 띄운 실행, 같은 시각 다른 작업 확인
5	**개발자**	**폭주는 이미 끝남. 부하를 다시 만들어 재현 후 측정**

여러 도구, 5단계. 원인 코드는 재현부터.

해결

띄우고, 지켜보고, 돌아보는 일을 번호 하나로 잇습니다

스케줄러가 실행마다 번호를 붙이면, 그 실행이 띄운 서비스의 자원·업무 지표·함수 기록이 같은 번호, 같은 시간축으로 쌓입니다.

1 띄운다

플로우가 서비스를 기동하며
실행 번호를 발급합니다.

지금 배치 스케줄러

2 지켜본다

서버 자원, 업무 지표, CPU를 쓴 함수를
같은 번호로 수집합니다.

지금 서버 관제 + 업무 관제 (함수 기록 없음)

3 돌아본다

지난 구간을 다시 펼쳐 함수까지 보고
SQL·CSV로 꺼냅니다.

지금 제품 화면 안에서만

실행 번호 `01JBQ7…` 스크립트가 끝나도, 서비스가 끝날 때까지 같은 번호

설치 구성

에이전트 서버마다 1
실행·수집·함수 기록

→

매니저 1
스케줄링·저장·화면

→

통합 개방 DB
실행 이력·자원·업무 지표·
함수 기록을 한곳에

1 · 1 · 1
실행 파일·화면·방화벽 포트

같은 실행 파일 eum을 역할만 바꿔 띄웁니다. 연결은 에이전트 → 매니저 한 방향입니다.

완성도 · 약속과 결과

기획서에서 약속한 것은 모두 동작합니다

기획서 14쪽의 세 흐름과 "시간이 남으면" 항목을 모두 구현했고, "이번에 안 함"으로 뺐던 로그인·권한·TLS까지 넣었습니다.

반드시 완성할 세 흐름 3 / 3

기획서 14쪽 · 15:00까지 완성할 흐름

- **실행**
  선후행·재시도 플로우로 서비스 기동
- **연결**
  기동 스크립트가 끝나도 같은 번호로 조회
- **활용**
  지난 구간 비교, 수집 데이터를 SQL로

시간이 남으면 하려던 것 4 / 4

기획서 14쪽 · 넓히기 항목

- **업무 지표 조건 실행**
  정산은 체결 대기 0건 뒤에 시작
- **함수 비교 표**
  A·B 구간의 함수별 비중 변화
- **임계치 알림**
  타일 기준을 넘으면 5초 주기로
- **실행 강제 종료**
  운영 역할부터, 감사 기록에 남음

약속보다 더 만든 것 추가

시연 뒤 나온 요구를 반영

- **로그인·역할 3개·감사 기록·TLS**
  기획서에서는 "이번에 안 함"
- **종료 보고서**
  서비스가 죽은 직접 원인
- **장애 위험도**
  같은 DB로 학습하는 참고 지표
- **편집기·사용자 보드·업무 흐름**
  운영자가 화면에서 직접 구성

남은 것 이중화·휴일 캘린더·자바 함수 추적·잠금/디스크 대기 분석은 기획서와 같이 다음 단계입니다.

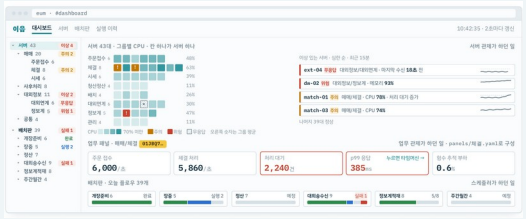
완성도 · 시안과 실물

기획서의 화면 시안이 실제 화면이 됐습니다

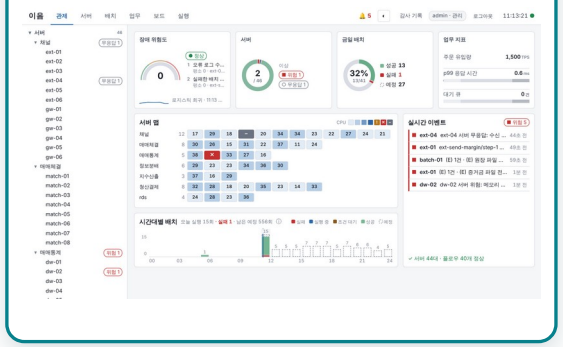
기획서 8~11쪽의 시안 네 장을 같은 구성으로 구현했습니다. 아래 줄은 모두 실제로 동작하는 화면을 캡처한 것입니다.

대시보드 기획서 8쪽

시안



실제 화면

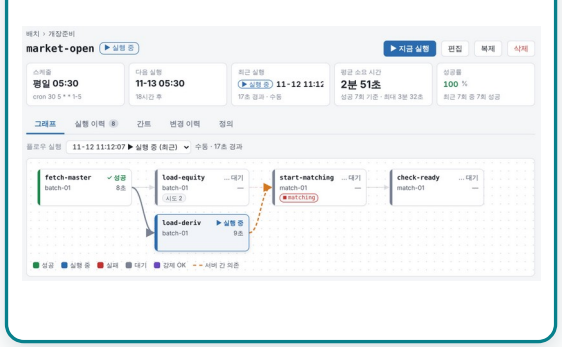


배치판·플로우 그래프 9쪽

시안



실제 화면



타임머신 10쪽

시안



실제 화면

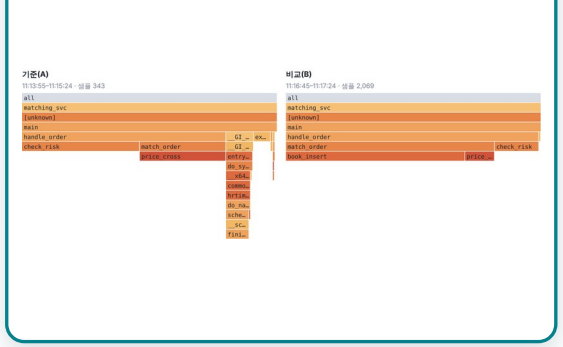


플레임 그래프 11쪽

시안



실제 화면



완성도 · 시연

시연: 주문 폭주 한 건을 처음부터 끝까지

영상에서 보실 여섯 장면입니다. 아래 화면은 같은 장면을 발표 준비 때 다시 실행해 캡처한 것입니다.



1 한 화면 관제

서버 46대·플로우 41개를 한 화면에



2 플로우 실행

병렬·합류·서버 넘기·재시도(시도 2)



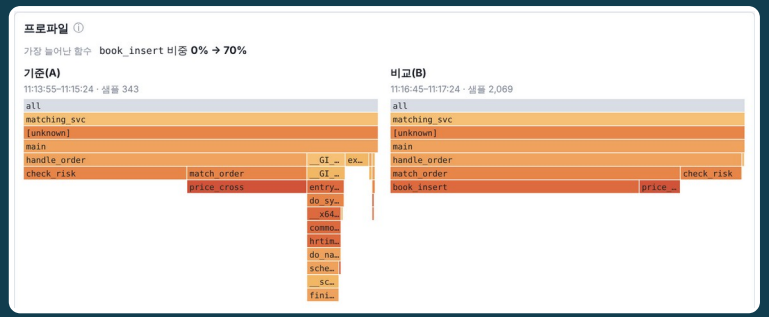
3 주문 폭주

유입 1,500 → 6,000건/초, 대기 큐 3,785건



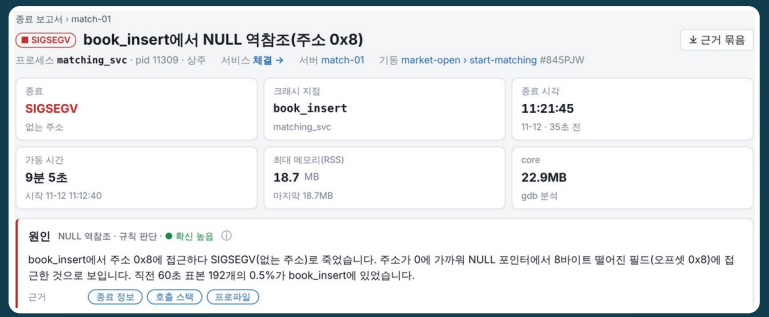
4 위험 감지

타일 알림에 이어 장애 위험도 100



5 함수까지

평시와 지연 구간 비교: book_insert 0% → 70%



6 죽은 원인

크래시 6초 뒤, 원인이 적힌 종료 보고서

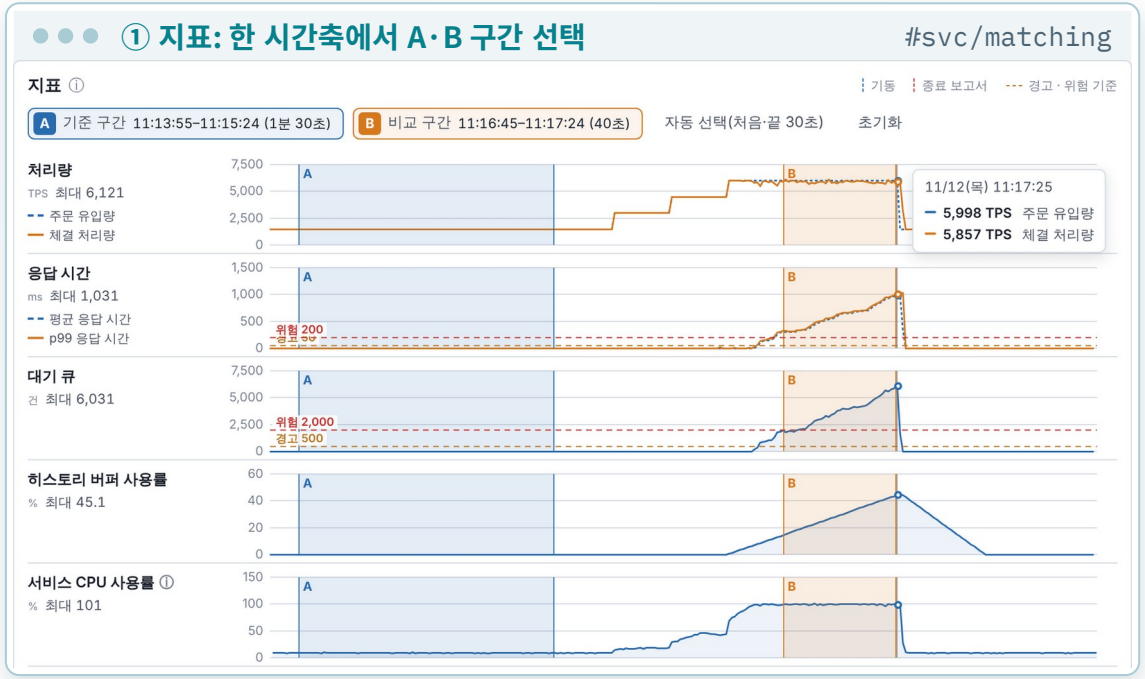
완성도 · 핵심 장면

지나간 폭주를 다시 펼쳐, CPU를 쓴 함수까지 내려갑니다

주문이 초당 1,500건에서 6,000건으로 몰린 구간입니다. 번호로 모은 기록을 한 시간축에 놓고, 평시(A)와 지연 구간(B)을 끌어서 비교했습니다.

① 지표: 한 시간축에서 A·B 구간 선택

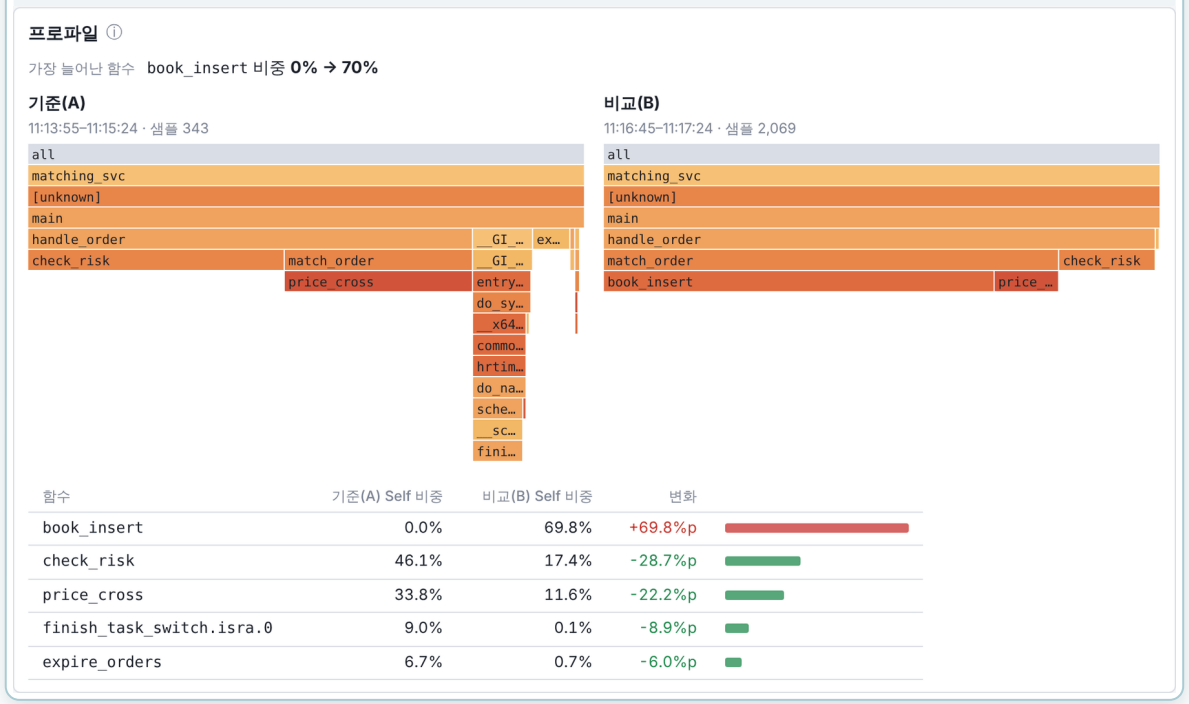
#svc/matching



유입 **1,500→6,000건/초** | p99 응답 **0.6→1,031ms** | 대기 큐 **0→6,031건**

② 프로파일: 두 구간의 함수 비교

#svc/matching



결과 가장 늘어난 함수 **book_insert** · CPU 비중 **0% → 70%**

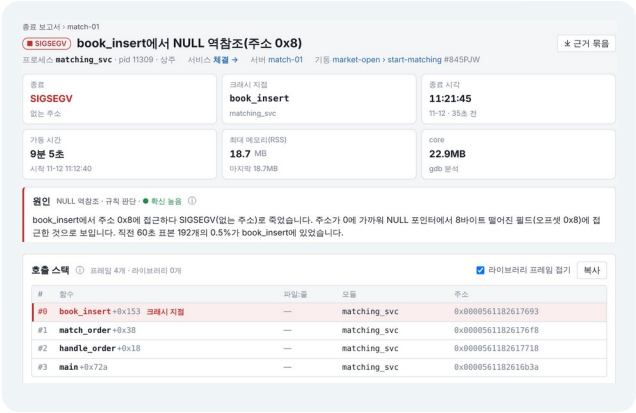
서버 증설보다 호가 등록 코드를 먼저 볼 근거가 재현 없이 나옵니다.

이음 · 화면은 실제 동작 캡처, 수치는 합성 부하로 다시 실행해 잰 값

완성도 · 더 만든 것

약속한 범위를 넘어, 운영에 필요한 것을 더했습니다

프로토타입 시연에서 나온 요구를 반영했습니다. 네 화면 모두 실제로 동작합니다.



종료 보고서

서비스가 죽은 직접 원인

서비스가 죽은 지 6초 뒤, 원인이 적힌 알림이 옵니다. core 파일은 서버 안에서 분석합니다.



장애 위험도

같은 DB로 학습하는 참고 지표

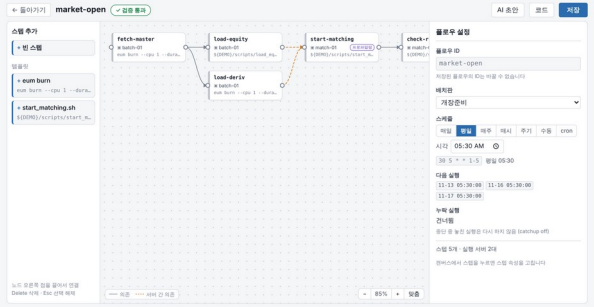
과거 장애 기록으로 로지스틱 회귀를 학습합니다. 폭주 51초 뒤 위험(99)에 올랐습니다.



사용자 보드·업무 흐름

어느 업무에서 막혔나

업무 패널 일곱 개를 상자와 화살표로 잇습니다. 보드는 YAML 파일 하나로 정의합니다.



화면에서 작업 구성

정의는 여전히 파일

편집기는 허용 목록 안에서 YAML 파일을 씁니다. 재실행과 강제 종료 같은 조작은 감사 기록에 남습니다.

이음 · 화면은 실제 동작 캡처, 수치는 합성 부하로 다시 실행해 잰 값

완성도 · 검증

동작은 테스트와 재실행으로 확인했습니다

테스트는 저장소 그대로 다시 돌렸고, 시연 장면은 처음부터 다시 실행해 화면과 수치를 얻었습니다(2026-11-12).

883
테스트 통과
하위 사례 포함 · 실패 0

39
패키지 통과
테스트가 있는 패키지 전부

9단계
개념 완결 E2E
실제 perf로 함수 표본까지

46대
시연 규모
그룹 7개 · 플로우 41개

gofmt·go vet 지적 0건. 환경은 리눅스 4코어, 매니저 1 + 에이전트 4 + 모의 서버 42대입니다. 서버 자원은 모의 값, 부하는 합성 주문입니다.

발표 준비 때 다시 실행해 확인한 장면

체결 폭주	유입 1,500 → 6,000건/초, p99 1,031ms, 대기 큐 6,031건
함수 비교	book_insert 비중 0% → 70% (표본 343개 → 2,069개)
시세 폭주	시세 지연 p99 638ms, rebuild_book 0% → 21%
정산 게이트	체결 대기가 남은 35초 동안 보류, 대기가 비자 바로 시작
크래시	SIGSEGV 6초 뒤 원인 알림, core는 서버 안에서 분석
권한	로그인 없음 401 · 조회 계정의 실행 403 · 다른 출처 403

업무 활용성 · 운영 환경

우리 운영 환경의 조건을 그대로 받아들입니다

폐쇄망, 방화벽, 기존 셸 스크립트와 C 서비스. 운영이 요구하는 조건에 맞춰 설계했고 시연 구성에서 확인했습니다.

운영 조건	이음의 방식	시연 구성에서 확인한 것
폐쇄망	외부 서비스·CDN 없이 동작합니다. 화면 라이브러리까지 실행 파일 안에 들어 있습니다.	**외부 의존성 4개, 빌드 단계 없는 화면**
방화벽	에이전트 → 매니저 한 방향으로만 연결합니다. 여는 포트는 매니저 1개입니다.	**서버 46대 구성이 포트 1개로 동작**
기존 배치 자산	셸 스크립트를 그대로 실행합니다. 스크립트가 nohup·setsid로 띄운 C 서비스도 같은 번호로 추적합니다.	**체결·시세 서비스(C, -O2 빌드) 추적**
서버 부담	함수 추적은 표본 방식(초당 49회)입니다. 추적 부하를 숨기지 않고 화면에 띄웁니다.	**부하 실측 약 3% (코어 1개 기준)**
설치·자원	실행 파일 1개(24MB)와 YAML 파일입니다. 저장은 파일 DB라 별도 DB 서버가 없습니다.	**46대 구성 전체가 메모리 약 120MB**

설치 순서 ① 매니저 1대에 실행 파일 ② 서버마다 같은 파일을 에이전트로 ③ 플로우·패널을 YAML로

업무 활용성 · 통제

운영 도구에 필요한 통제를 갖췄습니다

누가 무엇을 할 수 있는지, 서버에서 무엇이 실행될 수 있는지, 누가 무엇을 했는지를 각각 막고 남깁니다.

화면: 누가 무엇을 할 수 있나

역할 3개

로그인과 역할 3개(조회·운영·관리)를 둡니다. 다른 출처의 변경 요청과 반복된 로그인 실패를 막습니다.

서버: 무엇이 실행될 수 있나

호스트별 토큰 | TLS

매니저와 에이전트가 명령 허용 목록을 각각 검사합니다. 매니저가 뚫려도 서버는 정해진 명령만 실행합니다.

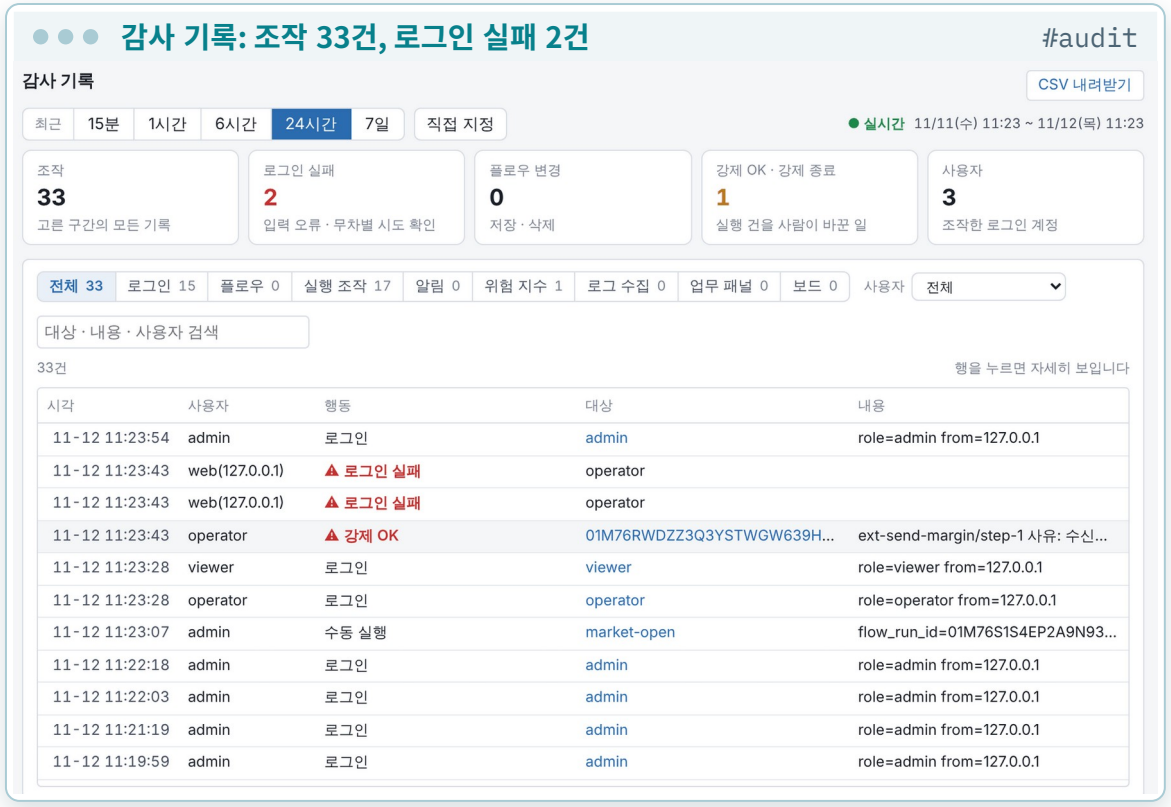
기록: 누가 무엇을 했나

감사 기록

실행·재실행·강제 종료·강제 OK와 로그인 실패까지 감사 기록 한 목록에 남고, CSV로 내려받습니다.

다시 확인

- 로그인 없이 호출 **401**
- 조회 계정의 실행 **403**
- 다른 출처의 요청 **403**



이음 · 화면은 실제 동작 캡처, 수치는 합성 부하로 다시 실행해 잰 값

업무 활용성 · 우리 업무 붙이기

새 업무는 코드 수정 없이 파일 하나로 붙입니다

정의의 기준은 파일(YAML)입니다. 화면 편집기는 그 파일을 쓰는 도구라서, 깃으로 검토하고 관리하던 방식이 그대로 통합니다.

플로우 `flows/*.yaml`

선후행·재시도·조건 실행을 정의합니다. 화면 편집기로도 씁니다.

업무 패널 `panels/*.yaml`

업무마다 볼 지표와 집계, 경고·위험 기준을 적습니다.

서비스 `services/*.yaml`

어떤 프로세스가 그 서비스인지, 언제 떠 있어야 하는지 적습니다.

사용자 보드 `dashboards/*.yaml`

위젯 15종을 격자에 놓습니다. 벽면 표시도 됩니다.

화면에서 고쳐도 결과는 같은 파일입니다. 저장과 삭제는 감사 기록에 남습니다.

```yaml
# panels/02-매매체결.yaml (실제 파일에서 발췌)
id: matching
title: 매매체결
service: matching
tiles:
  - { title: p99 응답 시간, metric: biz.order.latency_p99_ms,
      agg: max, unit: ms, warn: 50, crit: 200 }
  - { title: 대기 큐, metric: biz.order.queue,
      agg: sum, unit: 건, warn: 500, crit: 2000 }
```

이 몇 줄로 타일·기준선·알림이 생깁니다. 재시작 없이 [다시 읽기]로 반영됩니다.

업무 지표를 붙이는 세 가지 길

프로그램이 보냄
UDP로 이름과 값 한 줄

로그에서 뽑음
화면에서 숫자 뽑기 설정

스크립트로 보냄
고칠 수 없는 프로그램용

업무 활용성 · 적용 순서

기존 도구를 그대로 둔 채 내일부터 시작할 수 있습니다

스케줄러를 바꾸지 않아도 관측부터 시작할 수 있습니다. 병행 운영으로 실제 수치를 얻은 뒤 대체할 범위를 정합니다.

1단계 관측부터

검증계 서버 그룹 하나에 에이전트를 올립니다. 서버·서비스·업무 지표·함수 기록을 한 화면에서 봅니다.

기존 스케줄러·관제는 그대로

→

2단계 배치 병행

개장 준비 플로우 하나를 이음 YAML로 옮겨 기존 스케줄러와 나란히 돌리고 결과를 비교합니다.

같은 시나리오로 조사 시간 실측

→

3단계 대체 범위 결정

병행 결과로 대체할 범위와 비용을 계산합니다. 이중화·휴일 캘린더 등 남은 항목을 채웁니다.

금액은 이 단계에서 계산

지금 준비된 것

- 실행 파일 1개 설치, 폐쇄망 동작
- 로그인·역할·감사 기록·허용 목록·TLS
- C 서비스 함수 추적(운영 서버 perf 사용 가능)

적용 전에 채울 것

- 이중화, 휴일 캘린더, 사내 인증(SSO) 연결
- 이음 밖에서 뜬 서비스 관측: 실서버 확인 필요
- 자바 함수 추적, 운영 서버에서 부하 다시 측정

이음 · 화면은 실제 동작 캡처, 수치는 합성 부하로 다시 실행해 잰 값

개선 효과 ① 장애 조사

장애 원인 추적이 5단계에서 2단계로 줄어듭니다

도구를 바꿔 가며 시각을 맞추던 일이 타일 클릭과 구간 선택 두 번으로 줄고, 원인 코드를 찾으려고 부하를 재현할 필요가 없어집니다.

지금 도구 4곳 · 5단계 · 원인 코드는 재현한 뒤에

1. **업무 관제** — 지연 확인
2. **운영 담당** — 서버·프로세스 확인
3. **서버 관제** — "CPU를 다 씀"에서 멈춤
4. **스케줄러** — 띄운 실행 확인
5. **개발자** — 부하 재현 후 측정

이음 화면 1개 · 2단계 · 재현 없음

1. **업무 타일 클릭** (화면 응답 0.13초) — 빨갛게 변한 타일을 누르면 그 서비스의 타임머신이 열립니다.
2. **평시·지연 구간 선택** (화면 응답 0.19초) — 차트에서 두 구간을 끌면 함수별 CPU 비중 비교가 나옵니다.

가장 늘어난 함수

`book_insert` 0% → 70%

함께 잰 값

5초 기준 초과 알림 주기
6,000건/초가 된 지 16초 뒤 첫 알림

6초 크래시 → 원인 보고
죽은 함수와 호출 스택 포함

10초 함수 표본 저장 단위
늘 저장하므로 지나간 구간도 조회

단계 수는 절차 기준입니다. 사람이 쓰는 조사 시간은 병행 운영에서 같은 시나리오로 잽니다.

이음 · 화면은 실제 동작 캡처, 수치는 합성 부하로 다시 실행해 잰 값

개선 효과 ② 구조와 비용

도구 셋이 하나가 되어 관리할 것이 줄어듭니다

제품마다 따로이던 계약·관리 서버·화면·저장소가 하나로 줄어듭니다. 금액은 약속하지 않고, 줄어드는 항목과 늘어나는 항목을 함께 놓았습니다.

관리 대상	지금	이음
제품	3개 (스케줄러·서버 관제·업무 관제)	**실행 파일 1개**
화면	도구마다 따로	**1개**
저장소	도구마다 따로, 형식은 제품이 정함	**DB 1개, 공개 스키마**
방화벽	제품마다 따로	**포트 1개, 한 방향**
도구 사이 연결	사람이 시각·서버 이름으로 맞춤	**처음부터 같은 번호**

금액은 병행 운영에서 대체할 범위와 투입 인력이 정해진 뒤 계산합니다.

줄어드는 비용

- 겹치는 상용 제품의 라이선스·유지보수
- 제품별 관리 서버와 저장소
- 도구 사이 연동 개발
- 장애 조사에 쓰는 시간

늘어나는 비용

- 유지보수·기능 추가 인력
- 전환과 병행 운영
- 저장 공간, 보안 점검

개선 효과 ③ 데이터

한 번 보고 끝나던 운영 기록이 분석 재료가 됩니다

실행 이력·서버 자원·서비스 자원·업무 지표·함수 기록 다섯 가지가 공개 스키마로 한 DB에 있습니다. 읽기 전용 SQL과 CSV로 바로 꺼냅니다.

```
$ deploy/demo.sh sql
-- slow-functions.sql
leaf          | normal_pct | slow_pct
--------------+------------+---------
book_insert   |       11.1 |     69.1
check_risk    |       45.1 |     18.2
price_cross   |       29.3 |     11.3
expire_orders |        5.7 |      0.8
```

실제 실행 결과의 위 네 줄입니다. 여기서 평시는 p99가 50ms 이하인 모든 구간이라, 초당 1,500건 구간만 고른 타임머신의 A 구간(0%)보다 높게 나옵니다.

용량 판단

업무 지표 + 자원

유입량별 지연 조회

코드 개선

업무 지표 + 함수

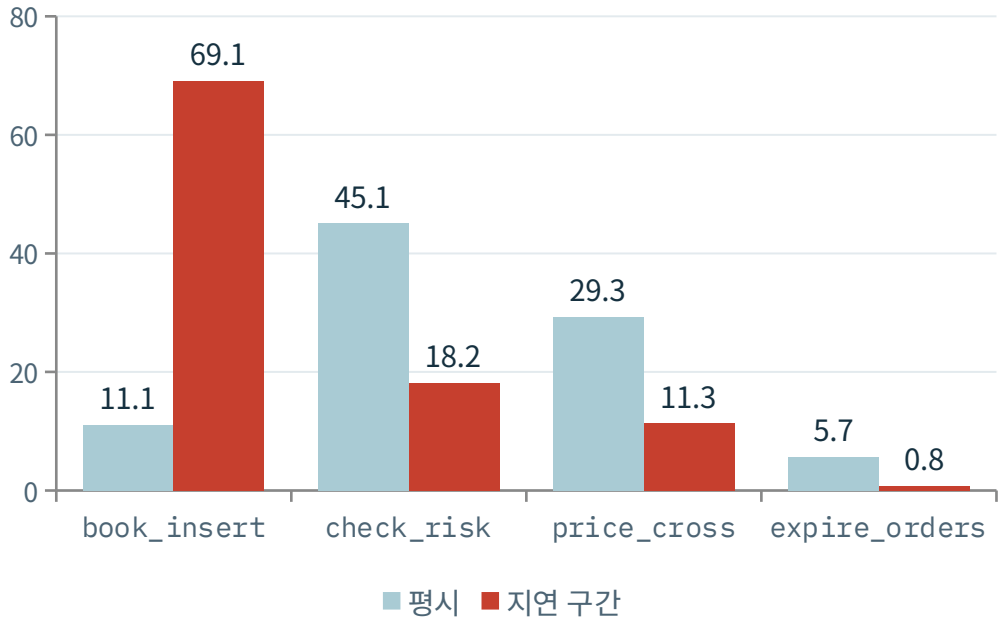
지연 때 커지는 함수 찾기

위험 예측

장애 기록 + 지표

학습 데이터를 CSV로

함수별 CPU 비중: 평시와 지연 구간 (%)



왼쪽 질의 결과를 그대로 그렸습니다.

개선 효과 · 종합

효과는 세 가지이고, 근거는 실제로 잰 값입니다

지금과 이음을 같은 항목으로 놓았습니다. 마지막 열은 그 차이를 무엇으로 확인했는지입니다.

항목	지금	이음	무엇으로 확인했나
장애 조사	도구 4곳, 5단계. 원인 코드는 재현한 뒤에	**화면 1개, 2단계. 재현 없이 함수까지**	폭주 재실행: book_insert 0% → 70%, 화면 응답 1초 미만
장애 감지	도구마다 따로 확인	**기준 초과 5초 주기 알림, 크래시 원인 보고**	알림 시각 기록: 첫 알림 16초 뒤, 크래시 원인 6초 뒤
관리 대상	제품 3 · 화면 3 · 저장소 3	**실행 파일 1 · 화면 1 · DB 1 · 포트 1**	서버 46대 · 플로우 41개를 한 구성으로 실행
데이터 활용	제품 화면·보고서 안에서만	**운영 데이터 5종을 SQL·CSV로 직접**	질의 3개 실행 결과, CSV·접힌 스택 내보내기
비용	제품 단위로 지출	**줄고 느는 항목을 함께 제시**	금액은 병행 운영 뒤에 계산

한계 수치는 합성 부하와 시연 환경 기준입니다. 운영 수치는 병행 운영에서 같은 방법으로 다시 잽니다.

만든 방법

AI는 제품이 아니라 만드는 방법에 썼습니다

사람은 계약과 검수를, AI 세션은 구현을 맡았습니다. 계약을 먼저 고정하고 테스트로 수정을 보호해 여러 세션이 동시에 일했습니다.

사람 2명

계약과 작업카드를 쓰고
결과를 검수합니다

→

오케스트레이터 1

의존 순서에 따라 카드를
세션에 배정합니다

→

워커 세션 4

작업 폴더 4개에서 카드를
나란히 구현합니다

→

테스트

계약 스위트·표 테스트·
E2E가 수정을 지킵니다

47장 작업카드

보충 카드 9묶음을 더함

5.9만 줄 Go 코드

그중 테스트 2.6만 줄

1.8만 줄 화면 JS

빌드 단계 없음

4개 외부 의존성

나머지는 표준 라이브러리

제품 안의 AI ① 장애 위험 지수

같은 DB의 장애 기록과 지표로 학습하는 로지스틱 회귀입니다. 판단을 대신하지 않는 참고 지표입니다.

제품 안의 AI ② LLM 연결 자리

자연어 작업 초안과 종료 보고서 참고 초안에 자리를 두었습니다. 사내 LLM을 연결하면 켜집니다.

이음 · 화면은 실제 동작 캡처, 수치는 합성 부하로 다시 실행해 잰 값

한계와 다음

아직 안 되는 것과 다음에 할 일

한계는 숨기지 않고 화면과 문서에 적어 두었습니다. 다음 단계는 병행 운영으로 실제 수치를 얻는 것입니다.

지금의 한계

⚠ **함수 기록은 CPU를 쓴 곳만 봅니다**
잠금·디스크 대기로 느린 경우는 범위 밖

⚠ **추적 부하가 목표보다 큽니다**
실측 약 3%, 목표는 1% 미만

⚠ **시연 수치는 합성입니다**
서버 자원은 모의 값, 부하는 합성 주문

⚠ **운영에 필요한 몇 가지가 없습니다**
이중화, 휴일 캘린더, 사내 인증 연결

다음 단계

1. **검증계 서버 그룹에서 관측 병행**
기존 도구는 그대로 두고 에이전트만

2. **같은 시나리오로 다시 측정**
사람의 조사 시간, 운영 서버의 추적 부하

3. **대체할 범위와 비용 계산**
줄어드는 항목과 늘어나는 항목을 금액으로

4. **확장 붙이기**
사내 LLM 연결, 자바 함수 추적, 이중화

보여 드린 것은 수치가 아니라 경로입니다. 띄우고, 수집하고, 번호로 잇는 일이 실제로 동작합니다.

맺음

사건은 하나, 도구도 하나

띄우고, 지켜보고, 돌아보는 일을 번호 하나로 이었습니다.

복잡도와 관리 비용

1 · 1 · 1

실행 파일·화면·방화벽 포트

장애 조사

5 → 2단계

재현 없이 CPU를 쓴 함수까지

데이터 활용

5종 → SQL

운영 데이터를 직접 엮어 분석

감사합니다. 질문 받겠습니다.

실행 번호 **01JBQ7…** 실행 → 연결 → 활용

부록 · 질문과 답

예상 질문 ① 추적과 수치

Q 서버 관제의 top 기록으로도 찾을 수 있지 않나요?

top 기록은 "체결 프로세스가 CPU를 다 썼다"까지입니다. 서버를 늘릴지 코드를 고칠지는 알 수 없습니다. 이음은 그 프로세스를 띄운 실행, 같은 시각의 업무 지표, CPU를 쓴 함수까지 잇습니다. 지나간 폭주도 표본이 남아 있습니다.

Q 운영 서비스에 프로파일러를 붙이면 느려지지 않나요?

표본 방식(초당 49회)이고 지정한 서비스만 봅니다. 이번에 잰 추적 부하는 코어 1개의 약 3%였고, 그 값을 화면에 그대로 띄웁니다. 표본 주기를 낮추면 더 줄어듭니다.

Q 시연 수치는 만든 값 아닌가요?

체결·시세 서비스는 합성 주문으로 부하를 만드는 시연용 C 프로그램입니다. 보여 드리는 것은 수치가 아니라 경로입니다. 띄운 서비스를 번호로 추적하고, perf로 함수를 모아, 같은 DB에 쌓는 과정이 실제로 동작합니다.

Q 우리 C 프로그램에서도 함수 이름이 나오나요?

실행 파일에 심볼이 남아 있으면 됩니다. 운영과 같은 조건(-O2, 프레임 포인터 없음)으로 빌드한 C 프로그램에서 호출 경로를 복원했습니다. 자바는 프로파일러 구현을 추가하는 확장입니다.

부록 · 질문과 답

예상 질문 ② 선택과 운영

Q 오픈소스를 조합하면 되지 않나요?

그렇게도 묶을 수 있습니다. 다만 스케줄러·모니터링·프로파일러 사이마다 연동 지점이 생기고, 그 연동 코드와 저장소 셋은 우리가 관리합니다. 이음은 실행·자원·업무·함수를 같은 번호, 같은 DB로 처음부터 묶었습니다.

Q 자체 개발은 유지보수 비용이 들지 않나요?

맞습니다. 그래서 줄어드는 비용과 늘어나는 비용을 함께 놓았습니다. Go 한 언어, 외부 의존성 4개, 테스트 883건으로 고치는 비용을 낮췄고, 금액은 병행 운영 뒤에 계산합니다.

Q 화면에서 아무 명령이나 돌릴 수 있지 않나요?

정의의 기준은 YAML 파일입니다. 편집기에서 새로 넣은 명령은 허용 목록을 통과해야 저장되고, 에이전트가 같은 목록을 한 번 더 검사합니다. 저장과 삭제는 관리 역할만 할 수 있고 감사 기록에 남습니다.

Q 몇 대까지 감당하나요?

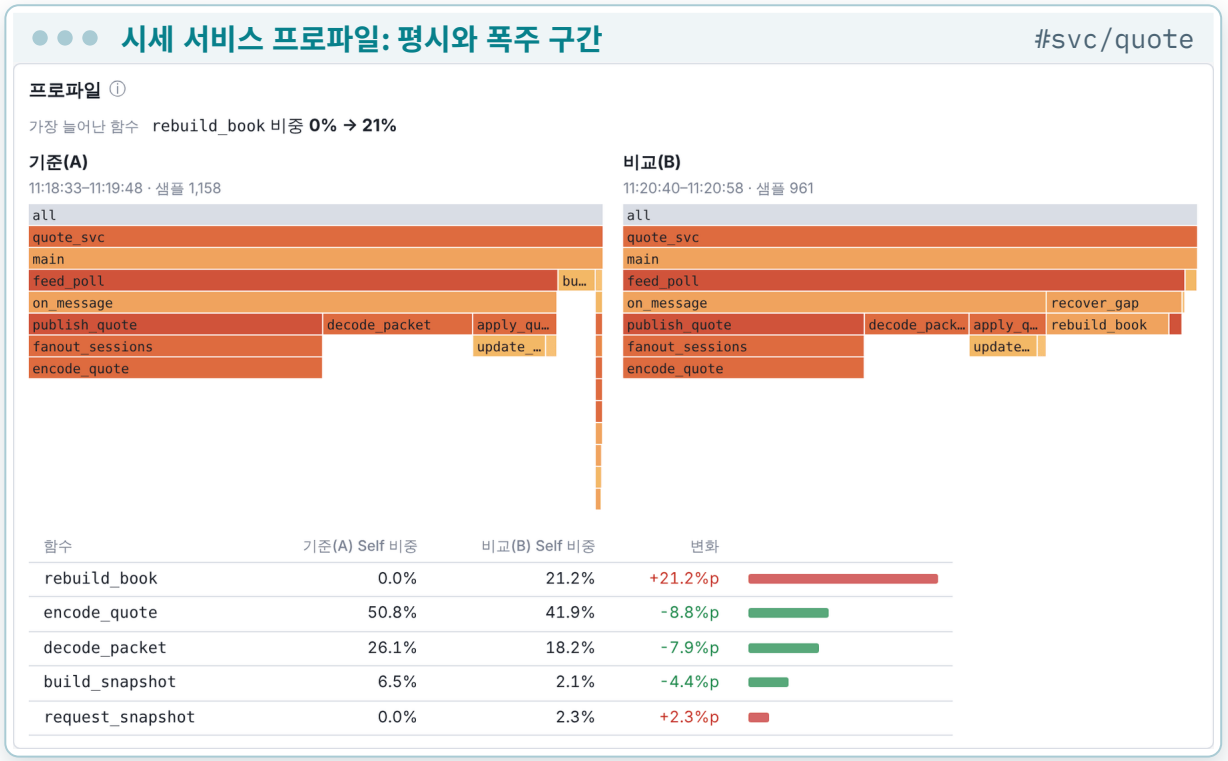
화면은 그룹 트리와 서버 맵이라 수백 대도 한 화면에 들어갑니다. 먼저 한계에 닿는 것은 저장 용량이고, 그때는 저장소 구현만 시계열 DB로 바꿉니다. 바뀌는 범위는 저장 패키지 하나입니다.

부록 · 두 번째 업무

다른 업무에서도 같은 길로 원인이 나옵니다

따로 도는 시세 서비스에 수신이 초당 8,000건으로 몰린 장면입니다. 체결과는 서버도, 밀리는 원인도, 함수도 다릅니다.





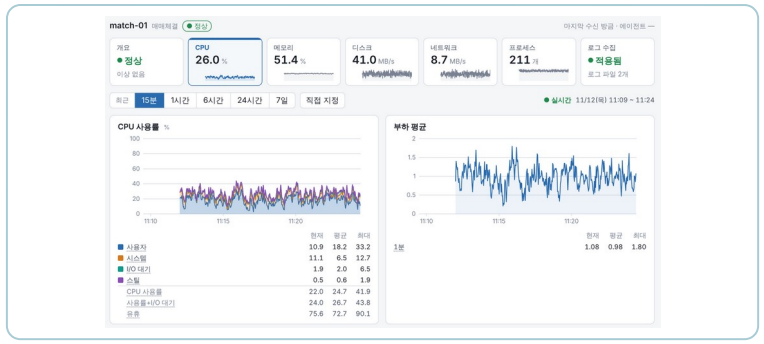
수신 **2,000→8,000건/초** | 시세 지연 p99 **638ms** | 수신 대기 **최대 4,089건**

결과 평시에 없던 함수 **rebuild_book** · CPU 비중 **0% → 21%** 수신 버퍼가 넘쳐 스냅샷 복구를 되풀이한 것이 원인. 체결 쪽 알림은 없었습니다.

이음 · 화면은 실제 동작 캡처, 수치는 합성 부하로 다시 실행해 잰 값

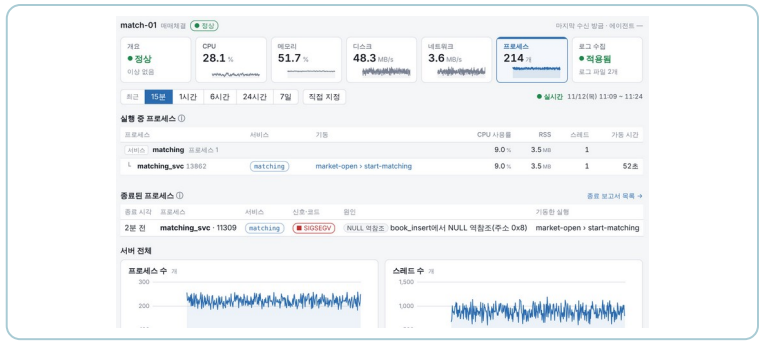
부록 · 화면 모음

그 밖의 화면도 모두 동작합니다



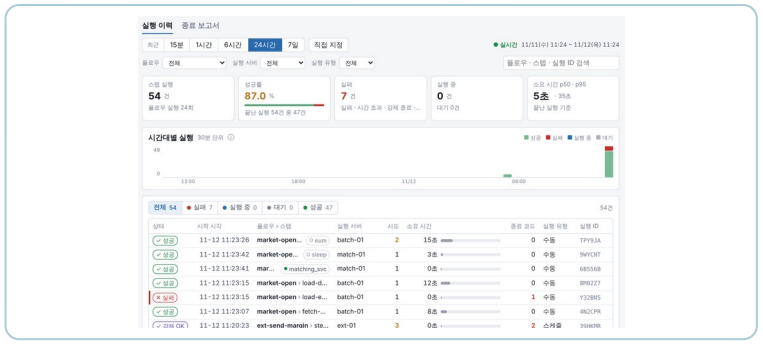
서버 상세

CPU·메모리·디스크·네트워크·프로세스, 7일 구간



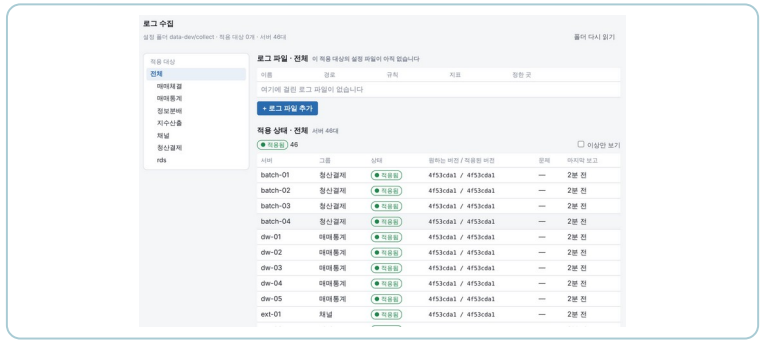
서버 프로세스

실행 중인 서비스와 종료된 프로세스의 원인



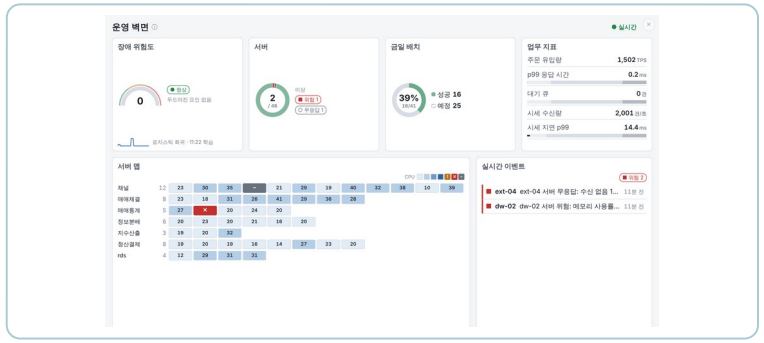
실행 이력

성공률, 소요 시간, 재시도와 강제 OK



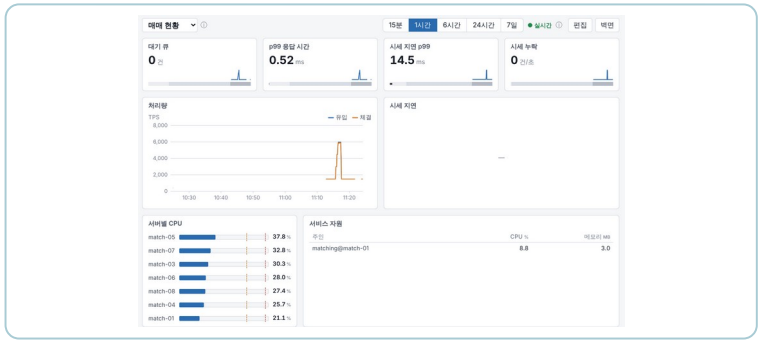
로그 수집

수집할 로그 파일과 규칙을 화면에서 설정



운영 벽면 보드

위 막대와 트리를 숨긴 벽면 표시



매매 현황 보드

패널 타일과 시계열을 섞은 사용자 보드

부록 · 수치의 출처

이 자료의 수치는 이렇게 쟀습니다

수치	값	어떻게 쟀나
테스트	**883건 통과 · 실패 0 (패키지 39개)**	go test ./... -v 의 통과 수, gofmt·go vet 지적 0건
체결 폭주	**p99 최대 1,031ms · 대기 큐 최대 6,031건**	demo.sh surge 뒤 서비스 타임머신의 최대값
함수 비중	**book_insert 0.0% → 69.8%**	A 11:13:55–11:15:24 표본 343 · B 11:16:45–11:17:24 표본 2,069
시세 폭주	**지연 p99 638ms · rebuild_book 0% → 21.2%**	demo.sh quote-surge, 서비스 로그와 타임머신
알림 시각	**첫 알림 16초 뒤 · 위험도 99는 51초 뒤**	6,000건/초 11:16:25 → 알림 11:16:41, 11:17:16
크래시	**원인 알림 6초 뒤**	demo.sh crash: 종료 11:21:45 → 알림 11:21:51
화면 응답	**0.13초 · 0.19초**	브라우저 자동 조작 5회의 중앙값
자원	**메모리 122MB · 실행 파일 23.7MB**	매니저 1 + 에이전트 46을 한 프로세스로 띄운 값

환경 2026-11-12, 리눅스 4코어 컨테이너, perf 6.8. 서버 46대 중 42대는 모의 서버이고 서버 자원은 모의 값입니다. 시연용 체결 서비스의 부하 상수(COST_SCAN)는 저장소 안내서의 절차대로 이 환경 CPU에 맞췄습니다(23 → 16).